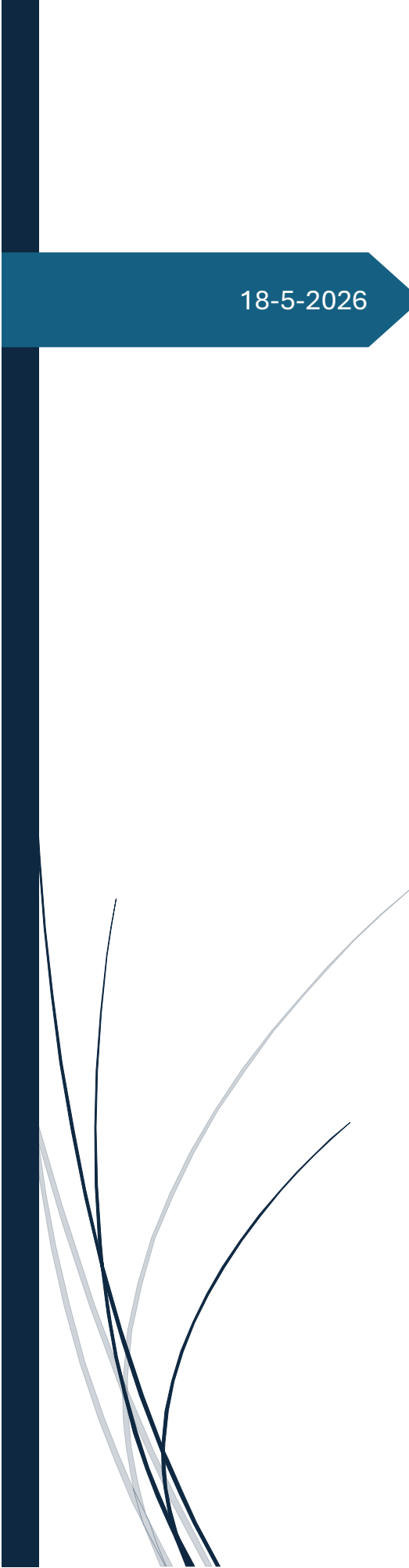




18-5-2026

Documentación

FRONTEND ANGULAR PARA
CONSUMO DE API REST DE
PRODUCTOS Y CARRITO DE
COMPRAS

Nombre del Estudiante: Daniel Altamirano
INSTITUTO SUPERIOR TECNOLÓGICO CEMLAD

Índice

1. Introducción
2. Objetivos
3. Marco Teórico
4. Desarrollo del Proyecto
5. Arquitectura del Frontend
6. Integración con la API REST
7. Manejo de Autenticación JWT
8. Navegación y Experiencia de Usuario
9. Manejo de CORS
10. Pruebas e Integración
11. Resultados Obtenidos
12. Conclusiones
13. Recomendaciones
14. Referencias Bibliográficas

1. Introducción

Las aplicaciones web modernas requieren una correcta integración entre el frontend y el backend para ofrecer experiencias rápidas, dinámicas y seguras. Actualmente, una gran cantidad de sistemas utilizan arquitecturas basadas en APIs REST para permitir la comunicación entre diferentes tecnologías.

En esta actividad se desarrolló una aplicación frontend utilizando Angular para consumir la API REST creada previamente en el Laboratorio 1 con Django REST Framework.

El sistema permite realizar autenticación mediante JWT, gestión de productos y administración de un carrito de compras conectado directamente con la API backend.

El proyecto permitió comprender el flujo completo de una aplicación web moderna, integrando frontend y backend mediante solicitudes HTTP, manejo de tokens y comunicación segura entre aplicaciones.

Además, esta práctica fortaleció conocimientos relacionados con Angular, consumo de APIs REST, autenticación y buenas prácticas de desarrollo frontend.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación frontend con Angular capaz de consumir una API REST de productos y carrito de compras utilizando autenticación JWT e integración completa con el backend desarrollado previamente.

2.2 Objetivos Específicos

- Implementar formularios de login y registro.
- Consumir endpoints REST desde Angular.
- Gestionar autenticación mediante JWT.
- Implementar CRUD de productos.
- Desarrollar un carrito de compras funcional.
- Utilizar servicios e interceptores siguiendo buenas prácticas.
- Integrar el frontend Angular con la API backend desarrollada en Django.

3. Marco Teórico

3.1 Angular

Angular es un framework frontend desarrollado por Google que permite construir aplicaciones web dinámicas utilizando TypeScript.

Angular trabaja bajo una arquitectura basada en componentes, permitiendo dividir la aplicación en partes reutilizables y organizadas.

Entre sus principales ventajas se encuentran:

- Arquitectura escalable.
- Uso de componentes reutilizables.
- Integración sencilla con APIs REST.
- Tipado fuerte mediante TypeScript.
- Sistema de routing.
- Manejo avanzado de formularios.

3.2 SPA (Single Page Application)

Una SPA o Single Page Application es una aplicación web que carga una sola página principal y actualiza el contenido dinámicamente sin recargar completamente el navegador.

Angular utiliza este enfoque para mejorar:

- La velocidad de navegación.
- La experiencia de usuario.
- La interacción dinámica.
- El rendimiento de aplicaciones modernas.

3.3 Consumo de APIs REST

El consumo de APIs REST consiste en realizar solicitudes HTTP hacia un servidor backend para obtener o enviar información.

En Angular esto se realiza mediante HttpClient, el cual permite ejecutar solicitudes como:

- GET
- POST
- PUT
- PATCH
- DELETE

La comunicación generalmente utiliza JSON como formato de intercambio de datos.

3.4 JWT en el Frontend

JWT (JSON Web Token) es un mecanismo de autenticación utilizado para validar usuarios mediante tokens.

Cuando el usuario inicia sesión correctamente, el backend genera un token JWT que es almacenado en el frontend.

Posteriormente, este token es enviado automáticamente en cada petición protegida.

Entre las ventajas de JWT se encuentran:

- Autenticación segura.
- Menor dependencia de sesiones.
- Integración sencilla con APIs REST.
- Escalabilidad.

3.5 Interceptors en Angular

Un interceptor en Angular permite modificar automáticamente las solicitudes HTTP antes de que sean enviadas.

En este proyecto se utilizó un `HttpInterceptor` para:

- Insertar automáticamente el token JWT.
- Proteger endpoints privados.
- Centralizar la autenticación.

Esto evita repetir código en múltiples componentes.

4. Desarrollo del Proyecto

4.1 Descripción General

El proyecto consiste en una aplicación frontend desarrollada con Angular que consume la API REST creada previamente utilizando Django REST Framework.

El sistema permite:

- Registro de usuarios.
- Inicio de sesión.
- Gestión de productos.
- Administración de carrito de compras.

- Navegación dinámica.
- Comunicación segura mediante JWT.

La aplicación fue estructurada siguiendo buenas prácticas de organización y separación de responsabilidades.

4.2 Tecnologías Utilizadas

Tecnología	Descripción
Angular	Framework frontend principal
TypeScript	Lenguaje utilizado por Angular
Angular CLI	Herramienta de generación del proyecto
HttpClient	Consumo de APIs REST
JWT	Autenticación segura
Bootstrap / Angular Material	Estilos visuales
Django REST Framework	Backend consumido

4.3 Organización del Proyecto

El proyecto fue organizado siguiendo una estructura modular.

Estructura principal:

```
core/  
  interceptors/  
  services/  
  models/
```

```
features/  
  auth/  
  products/  
  cart/
```

```
shared/  
  components/
```

Esta organización permite mantener el código limpio, escalable y fácil de mantener.

4.4 Modelos TypeScript

Todos los datos fueron tipados utilizando interfaces TypeScript.

Entre los modelos implementados se encuentran:

- Product
- Cart

- CartItem
- User

Esto permite reducir errores y mejorar el control de datos dentro de la aplicación.

4.5 Servicios

Los servicios fueron utilizados para centralizar todas las peticiones HTTP.

Servicios principales:

- auth.service.ts
- product.service.ts
- cart.service.ts

Los componentes no realizan solicitudes HTTP directamente, permitiendo mantener una arquitectura más organizada.

5. Arquitectura del Frontend

5.1 Arquitectura Basada en Componentes

Angular utiliza componentes reutilizables para construir interfaces dinámicas.

Cada módulo del sistema fue dividido en componentes independientes.

Componentes principales:

- Login.
- Registro.
- Listado de productos.
- Formulario de productos.
- Carrito de compras.
- Navbar.

Esta estructura facilita el mantenimiento y reutilización del código.

5.2 Flujo de Autenticación

El sistema implementa el siguiente flujo:

1. Usuario inicia sesión.
2. Backend genera JWT.
3. Frontend almacena token en localStorage.

4. Interceptor agrega automáticamente el token.
5. Backend valida autenticación.
6. Usuario accede a rutas protegidas.

Este flujo permite mantener sesiones seguras dentro de la aplicación.

6. Integración con la API REST

6.1 Comunicación Frontend-Backend

El frontend Angular consume directamente la API REST desarrollada con Django REST Framework.

La comunicación se realiza mediante solicitudes HTTP utilizando HttpClient.

Todas las operaciones CRUD del sistema interactúan directamente con el backend.

6.2 CRUD de Productos

El sistema implementa:

- Visualización de productos.
- Creación de productos.
- Edición de productos.
- Eliminación de productos.

Los formularios fueron desarrollados utilizando formularios reactivos de Angular.

6.3 Carrito de Compras

El carrito de compras permite:

- Agregar productos.
- Modificar cantidades.
- Eliminar productos.
- Visualizar subtotales.
- Calcular total general.
- Vaciar carrito.

Toda la información es gestionada desde la API backend.

7. Manejo de Autenticación JWT

La autenticación fue implementada mediante JWT.

Cuando el usuario inicia sesión:

- El backend devuelve un token.
- El token es almacenado en localStorage.
- El interceptor lo adjunta automáticamente en cada petición protegida.

Además, el sistema permite:

- Registro de nuevos usuarios.
- Login.
- Logout.
- Protección de rutas privadas.

El cierre de sesión elimina el token almacenado localmente.

8. Navegación y Experiencia de Usuario

La aplicación incluye una barra de navegación visible en todas las pantallas.

La navegación cambia dinámicamente dependiendo del estado de autenticación.

Funcionalidades de la Navbar

Usuario autenticado

- Acceso a productos.
- Acceso al carrito.
- Visualización del nombre del usuario.
- Opción de logout.

Usuario no autenticado

- Botón de login.
- Botón de registro.

Además, el sistema muestra indicadores visuales cuando un producto no posee stock disponible.

9. Manejo de CORS

Durante el desarrollo fue necesario configurar CORS para permitir la comunicación entre Angular y Django.

La API backend fue configurada utilizando django-cors-headers para aceptar solicitudes desde:

`http://localhost:4200`

Esto permitió que el frontend Angular pudiera consumir correctamente la API REST.

Sin esta configuración el navegador bloquea las peticiones por políticas de seguridad.

10. Pruebas e Integración

Las pruebas realizadas permitieron verificar el correcto funcionamiento del frontend conectado a la API real.

10.1 Evidencias Verificadas

Autenticación

- Registro correcto de usuarios.
- Inicio de sesión funcional.
- Almacenamiento de JWT.
- Logout exitoso.

Productos

- CRUD de productos operativo.
- Actualización correcta de información.
- Eliminación funcional.

Carrito

- Agregado de productos.
- Actualización automática del total.
- Eliminación de productos.
- Vaciado de carrito.

10.2 Integración Completa

La integración entre Angular y Django REST Framework funcionó correctamente permitiendo:

- Comunicación segura.
- Consumo de endpoints.
- Manejo de autenticación.
- Gestión dinámica de datos.

11. Resultados Obtenidos

El proyecto logró implementar exitosamente una aplicación frontend moderna conectada a una API REST funcional.

Entre los resultados principales se encuentran:

- Integración completa frontend-backend.
- Autenticación JWT funcional.
- CRUD de productos operativo.
- Carrito de compras dinámico.
- Navegación SPA.
- Arquitectura organizada.
- Uso correcto de servicios e interceptores.
- Interfaz funcional y usable.

Además, el proyecto permitió comprender mejor el flujo completo de aplicaciones web modernas utilizando Angular y APIs REST.

12. Conclusiones

El desarrollo de esta actividad permitió comprender el funcionamiento de aplicaciones frontend modernas utilizando Angular.

La integración con la API REST desarrollada previamente permitió experimentar el flujo completo entre cliente y servidor.

El uso de JWT facilitó la autenticación segura y el manejo de sesiones dentro del frontend.

Angular permitió organizar el proyecto de forma modular utilizando componentes, servicios e interceptores, mejorando considerablemente la mantenibilidad del sistema.

Una de las principales dificultades encontradas fue la integración inicial entre frontend y backend debido a configuraciones relacionadas con CORS y autenticación, sin embargo, estas fueron solucionadas mediante la configuración adecuada de la API y el interceptor.

Finalmente, esta práctica fortaleció conocimientos relacionados con Angular, consumo de APIs REST, autenticación, arquitectura SPA y desarrollo full stack.

13. Recomendaciones

- Implementar guards para proteger rutas privadas.
- Incorporar paginación en productos.
- Mejorar el diseño visual responsivo.
- Implementar manejo global de errores.
- Agregar pruebas automatizadas frontend.
- Incorporar despliegue utilizando Docker o servicios cloud.

14. Referencias Bibliográficas

Google. (2025). *Angular documentation*. <https://angular.dev/>

Mozilla Foundation. (2025). *HTTP overview*. <https://developer.mozilla.org/>

OpenAPI Initiative. (2025). *OpenAPI specification*. <https://www.openapis.org/>

Django Software Foundation. (2025). *Django REST framework documentation*. <https://www.django-rest-framework.org/>

TypeScript Team. (2025). *TypeScript documentation*. <https://www.typescriptlang.org/>

Freeman, A. (2023). *Pro Angular*. Apress.

Fielding, R. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

<https://chatgpt.com/>

Repositorios

Frontend Angular

<https://github.com/DannyelAlejandro/cemlad-teaching-angular>

Backend Django REST Framework

<https://github.com/Daniel-Metal/cemlad-teaching-django.git>